

Relatório Técnico — 3E Operações

Sistema de acompanhamento de pedidos, produção, ocorrências e expedição da **3E Ferro e Aço**, como camada operacional sobre o **Top Gerente** (legado, somente leitura).

- **Repositório:** 3e-operacoes (Next.js + TypeScript + Prisma/MySQL)
- **Data:** 2026-10-07
- **Documentos de apoio:** docs/requisitos.md , docs/regras-negocio.md , docs/arquitetura.md , docs/integracao-top-gerente.md , docs/perfis-permissoes.md , docs/testes.md , docs/execucao.md , docs/backlog.md , docs/go-live.md , docs/operacao.md , .specs/STATE.md .

1. Contexto e problema

A 3E registra os pedidos no **Top Gerente**, mas não consegue acompanhar a execução entre os setores. Um pedido pode conter itens de revenda, telhas e estruturas, cada um com um fluxo diferente; o controle é manual, em planilhas e impressos, e não há visão consolidada de produção, disponibilidade e entrega.

O problema não é uma tela isolada: é a **integração e sincronização** das informações comerciais (Top Gerente) com o acompanhamento operacional (novo sistema), respeitando entregas parciais e a dependência do legado.

2. Objetivos

- Importar pedidos do Top Gerente **sob demanda**, sem polling e sem escrever no legado.
- Organizar itens por setor e dar aos operadores uma fila de trabalho.
- Registrar execução, perdas/refugos/indisponibilidades e entregas.
- Controlar saldo pendente, disponibilidade e prazos.
- Aplicar perfis e permissões no servidor.
- Interface responsiva (celular, tablet, notebook, desktop), acessível (WCAG 2.1 AA).

3. Arquitetura

Aplicação fullstack Next.js (App Router) hospedável no Railway, organizada como **monolito modular**. O Top Gerente não é substituído nem acessado pelo navegador.

```
Usuário
|
v
Next.js (UI + Route Handlers)
|-- src/modules/* (domínio, independente de Next)
|-- Prisma -> MySQL próprio do app
|
+-- HTTPS autenticado --> Cloudflare Access/Tunnel --> Conector local 3E
|
|
+--> MySQL Top Gerente (SELECT only)
|
+--> callback HTTPS normalizado --> API do app
```

Princípios: regras de negócio puras e testáveis; dependências apontam para dentro do domínio; nenhuma escrita no legado; jobs idempotentes e auditáveis; mudança comercial importada nunca apaga histórico operacional.

Módulos (src/modules)

auth , usuarios , setores , pedidos , integracao , producao , ocorrencias , expedicao , prazos , indicadores .

4. Modelo de dados

O modelo lógico está em prisma/schema.prisma ; o conceitual em docs/modelo-dados/mer.md (MER/DER/classes).

Separação de conceitos-chave: - **Pedido/ItemPedido** = dados comerciais importados (snapshot do legado). - **Atividade** = unidade operacional (item × setor), base da fila. - **Execucao, Ocorrencia, Entrega** = registros operacionais do app. - **Usuario/UserRole/UserSector, Sector, CategorySectorMapping, MotivoOcorrencia, IntegrationJob/Event, AuditLog**.

Chaves de negócio: Order.legacyOrderKey = "emp:orc" , OrderItem.legacyItemKey = seq , mapeamento por categoriaLegado .

5. Integração com o Top Gerente (AWS)

- O banco topgerente (AWS RDS) é a **fonte somente leitura**; as tabelas usadas são orcamento (cabeçalho) e orcamento_itens (itens).
- O **conector local** (connector-local/) consulta o legado com SQL parametrizado e devolve o payload normalizado por **callback autenticado**.
- Fluxo: usuário informa o número do pedido → POST /api/integracao/pedidos cria um **job idempotente** e responde 202 → despacho assíncrono ao conector → conector lê o legado → callback → validação (Zod) + **upsert** de pedido/itens → job SUCCEEDED .
- Idempotência**: repetir a importação **não duplica**; atualiza dados comerciais e quantidade solicitada; histórico operacional é preservado.
- Divergência**: nova quantidade menor que o executado/entregue é registrada em AuditLog e evento do job.

6. Funcionalidades entregues

11 features, cada uma com spec → design → tasks → execute → verify em .specs/features/ .

#	Feature	Requisitos
1	Importação de pedido por número	RF001, RF017, HU06
2	Setores e classificação de itens	RF002, RF015
3	Produção e fila de atividades	RF003, RF004, RF007, RF018
4	Ocorrências (perda, refugo, indisponibilidade)	RF005
5	Disponibilidade e entregas	RF009, RF010, RF011
6	Usuários, perfis e permissões	RF013, RF014
7	Prazos e alertas	RF012
8	Indicadores e consulta	RF006, RF008
9	Frontend — fundação e chão de fábrica	UI
10	Frontend — telas dos perfis	UI
11	Qualidade e go-live	E2E, a11y, docs

7. Regras de negócio implementadas

- **Saldo pendente** = solicitado – executado conforme. **Perda/refugo não reduzem** a obrigação de produzir (decisão vigente).
- **Disponível para entrega** = executado – entregue.
- **Entrega parcial** não conclui o item; ao alcançar o executado, conclui.
- **Entrega acima do disponível** é bloqueada; exceção só do gerente, com **motivo obrigatório e auditoria completa**.
- **Unidades**: peça (inteiro), metro (2 casas), unidade.
- **Prazo opcional**: sem prazo = `SEM_PRAZO`, nunca atrasado; atraso só com prazo conhecido e ultrapassado sem conclusão.
- **Motivo obrigatório** para perda, refugo e indisponibilidade (lista fechada configurável).
- **Permissões**: Vendedor só consulta (e prazo); Operador atua nos seus setores; Gerente altera status/prioridade/prazo/exceções; Expedição registra entregas; Responsável pelo Sistema administra cadastros.

8. Processo de desenvolvimento

Adotou-se o `t1c-spec-driven`: para cada feature, **Specify** (critérios de aceitação em EARS) → **Design** → **Tasks** (atômicas, com testes e gate) → **Execute** (1 commit por tarefa) → **Verify** (verificador independente, autor ≠ verificador). Gates determinísticos: `validate_spec.py`, `validate_tasks.py`, `validate_state.py`. Onde o verificador reprovou, houve correção e re-verificação (features 1, 2, 6, 9).

Todo o histórico de decisões e passos está em `.specs/STATE.md` (`AD-001`, `AD-002`, handoff) e `docs/execucao.md`.

9. Testes e qualidade

- **568 testes unitários** (Vitest + Testing Library) e **14 testes E2E** (Playwright).
- **Cobertura de domínio**: ~96% de linhas, ~94% de branches.
- CI (GitHub Actions): `prisma:generate`, `lint`, `typecheck`, `typecheck:connector`, `test`, `build`.
- E2E valida **login** e **responsividade** (sem rolagem horizontal em 360/768/1024/1440 px).
- Verificação por **sensor de discriminação**: mutações de comportamento injetadas e mortas pelos testes.

10. Segurança

- Senha com `scrypt` + sal; sessão em **cookie httpOnly** assinado (HMAC).
- **Autorização no servidor** pela matriz de perfis (não só na interface).
- Credenciais apenas em **variáveis de ambiente** (`.env`, fora do versionamento).
- Conector lê o legado com **SQL parametrizado**; nenhuma escrita no Top Gerente.
- Logs sem senha/token/CPF completo.
- Recomendações: usuário **somente leitura** no legado e **rotação** da senha.

11. Como executar

Detalhado em `docs/operacao.md`. Resumo:

```
npm install
npm run prisma:generate
Copy-Item .env.example .env      # ajustar DATABASE_URL
npm run db:up                    # MySQL local (Docker) – opcional
npm run prisma:migrate          # ou prisma:deploy
npm run prisma:seed              # setores, motivos e usuários
npm run dev                      # http://localhost:3000
```

Para o ambiente AWS: o banco do app é o schema `3e_operacoes` ; o `topgerente` permanece somente leitura, acessado pelo conector.

12. Resultados (demonstração)

Validação operacional ponta a ponta com o pedido real **70435**:

Etapa	Resultado
Importar/reimportar 70435 (legado → app)	5 itens, sem duplicar
Classificar itens	atividades criadas (Telhas, Corte e Dobra)
Fila do operador	2 atividades
Execução 100 de 180	<code>IN_PROGRESS</code> , pendente 80
Ocorrência de perda 2 com motivo	registrada sem alterar o saldo
Entrega parcial 30 de 100 disponíveis	<code>PARCIAL</code>
Saldo do item	sol 180 · exec 100 · disp 70 · entr 30 · pend 80
Indicadores	painel por setor/status + produção por setor + prazo

13. Limitações e trabalhos futuros

- Migrations dependem de um MySQL de runtime (o app usa o schema `3e_operacoes` na AWS).
- E2E de fluxos com dados depende de banco; hoje cobre login e responsividade.
- Lista de motivos e mapeamento **categoria** → **setor** ainda pendentes de validação com o cliente (usados como sugestão/configurável).
- Significado de "pedido atendido" em Revenda com indisponibilidade pendente de definição.
- Máquina de estados do job: revisar o caso `FAILED` transitório seguido de callback tardio.
- Go-live: Cloudflare Tunnel/Access e conector no ambiente da 3E.

14. Conclusão

O sistema entrega a camada operacional sobre o Top Gerente: importa pedidos sob demanda (somente leitura no legado), organiza a produção por setor, registra execução, ocorrências e entregas, calcula saldo/disponibilidade/prazos, aplica perfis e permissões e oferece interface responsiva e acessível. O desenvolvimento seguiu um processo **orientado a especificação com verificação independente**, resultando em **155 commits**, **11 features verificadas** e **582 testes** (568 unitários + 14 E2E) verdes.

Anexos

- Especificações por feature: `.specs/features/<feature>/{spec,design,tasks,validation}.md`
- Diário de execução: `docs/execucao.md`
- Backlog revisado: `docs/backlog.md`
- Modelo de dados: `docs/modelo-dados/`
- Operação: `docs/operacao.md`
- Go-live: `docs/go-live.md`
- Decisões: `.specs/STATE.md` (`AD-001` , `AD-002`) e `docs/adr/0001-...md`